MACHINE LEARNING PROGRAM · CAPSTONE PROJECT

PROJECT DOCUMENTATION

An End-to-End Machine Learning Framework for AI-Driven Medical Symptom Routing

---

Team: Abdulrahman Jamal • Marwan Ayman • Hassan Raafat • Abdelbaset Haitham

Track: Machine Learning & Data Analysis

Program: NTI Machine Learning Training — Final Capstone Project

Date: 13 August 2026

Repository: GitHub - abdulrahmanjamal72/Smart-Medical-Diagnosis-System: An AI-powered healthcare system that predicts potential diseases based on medical data/symptoms and routes patients to the appropriate medical specialist. Developed as a final project for the NTI Machine Learning Training. · GitHub

Table of Contents

1. Executive Summary

This project delivers an end-to-end machine learning framework that predicts a patient's likelihood of diabetes, hypertension, and heart disease from initial intake data, and routes them to the appropriate specialist. The system combines a tuned multi-output Random Forest baseline with the TabPFN tabular foundation model inside a stacking ensemble, and is served through a local Streamlit application for interactive, real-time predictions.

The pipeline was built with a strong emphasis on data integrity: leakage columns were identified and removed, duplicate records were eliminated before every split, and out-of-fold predictions were used to build honest meta-features for the ensemble. A critical self-audit of the final near-perfect ROC-AUC scores is documented transparently in Section 8, distinguishing genuine methodological rigor from an artifact of the underlying dataset.

The result is a portfolio-quality, reproducible pipeline — from raw CSV to a deployed inference application — that satisfies every mandatory requirement of the Capstone Project guidelines.

2. Problem Statement & Motivation

2.1 The Real-World Problem

Global healthcare systems face a significant and well-documented triage inefficiency. Mis-triage error rates in emergency settings are reported between 15% and 33%, and a 2022 survey found that 100% of 41 surveyed countries reported severe Emergency Department overcrowding. In the United States alone, at least 12 million individuals suffer from diagnostic errors annually, with misdiagnosis accounting for 5–17% of adverse hospital events.

These inefficiencies translate directly into delayed treatment, avoidable clinician workload, and, in the worst cases, preventable harm. A system that can flag likely conditions early and route patients to the correct specialist has the potential to meaningfully reduce this burden.

2.2 Why This Problem

During ideation, the team weighed this healthcare routing system against a sports-analytics alternative (predicting football league standings). Healthcare was selected because it is a universal necessity that affects the entire population, whereas sports analytics serves a narrower audience. Optimizing triage can prevent disability and save lives — an outcome with an order of magnitude more societal value than a niche predictive application.

2.3 Project Objective

Build a machine learning system that, given a patient's intake vitals and clinical indicators, predicts the probability of diabetes, hypertension, and heart disease, and uses those predictions to recommend a specialist route (e.g., Endocrinology, Cardiology), packaged behind a simple local web interface.

3. Team & Roles

The project was structured around specialized roles to maximize rigor across the pipeline, in line with the guideline allowing groups of up to five members (this team comprised four).

Team Member	Primary Responsibility
Abdulrahman Jamal	Data Engineering & Exploratory Data Analysis
Marwan Ayman	Machine Learning Modeling & Algorithm Design
Hassan Raafat	Pretrained Model and Out-Of Fold Predictions
Abdelbaset Haitham	Ensemble Learning(Stacking) and Deployment

4. Dataset Description & Preprocessing Pipeline

4.1 Source Data

The project uses the “Data Warehouse Multiclass” dataset, comprising over 280,000 patient records across 39 attributes, including demographic information, vital signs, lab values, and lifestyle indicators, with three target conditions: diabetes, hypertension, and heart disease.

4.2 Data Integrity & Leakage Mitigation

An initial audit flagged data leakage — features that implicitly encoded the target variable, effectively giving the model the answer before prediction. Five leakage columns derived directly from the targets were removed, and over 2,200 duplicate records were eliminated to prevent the model from memorizing repeated patients.

4.3 Exploratory Data Analysis

- Target variables were binarized (0 = absence, 1 = presence) and their distributions plotted, revealing class imbalance that informed the later use of class-weighted loss functions.
- A Pearson correlation heatmap across numeric features identified highly redundant pairs (e.g., “Age in Years” vs. “Age Category”), which were pruned to reduce multicollinearity and dimensionality.

- Skewness was computed for all continuous clinical features; all scores fell between –0.5 and 0.5, indicating near-normal distributions, so no logarithmic or power transformations were required.

4.4 Feature Engineering & Encoding

- Binary encoding for Yes/No fields (e.g., smoker status).
- Ordinal encoding for ranked categorical levels (Low / Moderate / High → 0 / 1 / 2).
- One-hot encoding (with drop_first=True) for nominal variables, avoiding the dummy-variable trap.
- A final Spearman correlation pass across the fully encoded feature space confirmed that no new multicollinearity was introduced during encoding.

4.5 Reproducibility Artifact

The sanitized, leakage-free, encoded dataset was persisted to disk as a single CSV artifact, acting as the source of truth for every subsequent modeling phase.

5. Methodology & Technical Approach

5.1 Overview

The modeling approach progressed in three phases: a tuned multi-output baseline, an advanced ensemble incorporating a pretrained foundation model, and a rigorous self-audit of the results.

Phase 1 — Baseline: Multi-Output Random Forest

- A MultiOutputClassifier wrapping a class-weighted RandomForestClassifier was tuned via GridSearchCV over tree depth and estimator count.
- Models were scored with f1_macro to balance precision and recall under class imbalance.
- The optimized model was evaluated on a held-out test set and serialized with joblib as the baseline artifact.

Phase 2 — Advanced Ensemble: TabPFN + Stacking

- Prior to any split, exact duplicate feature rows were removed and the train/test partition was programmatically verified to have zero index overlap.
- Out-of-fold (OOF) predictions were generated for both the Random Forest and TabPFN (a pretrained tabular foundation model) using 5-fold Stratified K-Fold cross-validation, ensuring no base learner ever saw the data it predicted on.
- OOF probabilities from both base learners were concatenated into a meta-feature space, on which a Logistic Regression meta-model was trained — learning, per disease, how much to trust each base learner.

- Precision-Recall curves were used to derive a per-disease decision threshold that maximizes F1, reflecting the clinical reality that a missed diagnosis (false negative) is costlier than an unnecessary follow-up (false positive).

Phase 3 — Critical Self-Audit

Because the final ROC-AUC scores approached 1.0 — uncommon for real clinical data — the team ran a dedicated investigation (Section 8) rather than accepting the result at face value.

5.2 Technology Stack

Component	Tool / Library
Language	Python 3.x
Data manipulation	Pandas, NumPy
Classical ML	Scikit-Learn (RandomForestClassifier, MultiOutputClassifier, GridSearchCV, LogisticRegression)
Foundation model	TabPFN (pretrained tabular transformer)
Validation	Stratified K-Fold cross-validation, out-of-fold prediction
Ensembling	Stacking ensemble with a Logistic Regression meta-learner
Serialization	joblib
Deployment	Streamlit (local web application)
Version control	Git / GitHub

6. Model Evaluation Metrics & Results

6.1 Metrics Used

- F1-macro — primary metric during hyperparameter tuning, balancing precision and recall across imbalanced classes.
- Precision, Recall, and per-class classification reports — for the baseline Random Forest on Diabetes, Hypertension, and Heart Disease.
- ROC-AUC — primary metric for the final stacking ensemble, evaluated on a strictly unseen, duplicate-free test set.
- Precision-Recall curves — used to derive an F1-optimal decision threshold per disease, rather than defaulting to 0.5.

6.2 Results Summary

Model	Key Metric	Result
Baseline — Multi-Output Random Forest	f1_macro (tuned, 3 targets)	Established benchmark for ensemble comparison
Stacking Ensemble (RF + TabPFN)	ROC-AUC (held-out test set)	Approaching 1.0 across all three targets
Depth-3 Decision Tree (audit control)	ROC-AUC	0.81 — far below the full ensemble

The gap between the shallow control tree (0.81 AUC) and the full ensemble (≈1.0 AUC) is itself diagnostic evidence, discussed in Section 8: genuinely leaked features are usually exploitable by very simple models, whereas this result required a highly complex model — pointing instead to a subtler, dataset-level artifact.

7. System Architecture & Deployment

7.1 Architecture Overview

The production pipeline follows a simple, three-stage architecture: intake → inference → routing.

```
Patient Intake Form (Streamlit UI)
        |
        v
Preprocessing (encoding, scaling to match training pipeline)
        |
        v
Base Models: Random Forest  +  TabPFN
        |
        v
Meta-Model (Logistic Regression) -> calibrated probability per disease
        |
        v
Thresholding -> Specialist Recommendation (Cardiology / Endocrinology / ...)
```

7.2 Deployment Details

- Interface: a local Streamlit application (streamlit run app.py) exposing an intake form for patient vitals and clinical indicators.
- Inference: the serialized base models, meta-model, and per-disease decision thresholds (joblib artifacts) are loaded once at startup and reused across requests.

- Output: for each of the three target conditions, the app displays a calibrated risk probability and, where the tuned threshold is exceeded, a recommended specialist route.
- Environment: dependencies are pinned in requirements.txt; the app runs entirely locally with no external API calls, satisfying the local-deployment requirement.

7.3 Running the Application

```
# clone the repository
git clone https://github.com/<team>/ai-medical-symptom-routing.git
cd ai-medical-symptom-routing

# install dependencies
pip install -r requirements.txt

# launch the app
streamlit run app.py
```

8. Critical Investigation of Model Performance

A near-perfect ROC-AUC is uncommon in real-world clinical data, so the team treated the result as a hypothesis to be tested rather than a success to be reported outright.

8.1 Investigation Steps

- Feature ablation: direct clinical indicators (Glucose, HbA1c) were removed and the model re-evaluated. ROC-AUC remained at 1.000, ruling out leakage concentrated in a single obvious feature.
- Model-depth analysis: a shallow depth-3 decision tree achieved only 0.81 AUC, while near-perfect scores were exclusively reproducible with highly complex models — the opposite signature of typical single-feature leakage, which shallow models usually exploit easily.

8.2 Conclusion

The near-perfect scores are attributable to subtle, dataset-level statistical artifacts rather than genuine clinical separability or an obvious leakage bug. The source dataset appears to have been assembled from multiple distinct cohorts, with some fields likely imputed synthetically; complex models are able to detect the resulting “statistical fingerprint” of the original data sources, inflating separability. This is reported transparently as a dataset limitation, not a claim of clinical readiness.

8.3 Other Limitations

- The dataset is not a validated, prospectively collected clinical cohort, so performance is not evidence of real-world diagnostic accuracy.
- The system is a decision-support and routing aid, not a diagnostic device, and has not undergone any clinical or regulatory validation.

- TabPFN's computational cost and input-size constraints may limit scalability to substantially larger feature sets or record volumes without further engineering.

9. Challenges Encountered & Solutions

Challenge	Solution
Data leakage inflating early model performance	Systematic audit of all 39 columns; removal of 5 features derived directly from target labels
Duplicate patients contaminating train/test split	Explicit duplicate removal on the feature set prior to splitting, with a programmatic zero-overlap check
Class imbalance across all three targets	Class-weighted loss functions and f1_macro as the tuning metric instead of accuracy
Meta-model overfitting to base-model training errors	Out-of-fold predictions via 5-fold Stratified K-Fold cross-validation for both base learners
Inappropriate 0.5 default threshold for clinical risk	Per-disease threshold tuning via Precision-Recall curves optimizing F1
Unexplained near-perfect ROC-AUC	Dedicated ablation and model-complexity audit (Section 8) rather than accepting the metric at face value

10. Future Work & Improvements

- Validate the full pipeline on an independent, prospectively collected, non-synthetic clinical dataset to obtain a trustworthy estimate of real-world performance.
- Integrate with hospital Electronic Health Record (EHR) systems via API for direct intake, removing manual data entry.
- Extend deployment from a local Streamlit demo to a containerized, cloud-hosted service with authentication and audit logging suitable for a clinical pilot.
- Add model explainability (e.g., SHAP values) to the interface so clinicians can see which factors drove each recommendation.
- Expand the target condition set beyond diabetes, hypertension, and heart disease as more validated data becomes available.

11. Repository Structure & Documentation

The public GitHub repository is organized for clarity and reproducibility, per the Capstone code-quality requirements:

The README.md includes the project description, setup instructions, dependency list, and usage guidelines, and this document expands on each section in full detail for the final discussion session.

12. Capstone Deliverables Checklist

Deliverable	Status
Public GitHub repository with complete source code	✓Complete
README with description, setup, dependencies, usage	✓Complete
Presentation slides for the final discussion	✓Complete
Working Streamlit / Flask application	✓Complete (Streamlit)
LinkedIn post showcasing the project	✓Complete

13. Conclusion

This project demonstrates a complete, methodologically rigorous machine learning workflow — from raw, leakage-prone data to a deployed, threshold-calibrated stacking ensemble — applied to a problem of genuine real-world significance. Equally important, the team treated an unusually strong result with appropriate skepticism, running a dedicated audit rather than reporting the headline metric uncritically. Together with the accompanying GitHub repository, presentation, and live demo, this documentation is intended to serve as a portfolio-quality record of both the technical and communication skills developed throughout the program.